

AR Castle Defense

Joc mòbil de defensa de torres en Realitat Augmentada

1. Nom i descripció general de l'aplicació

AR Castle Defense és un joc de defensa de torres en Realitat Augmentada desenvolupat amb Unity 6 i AR Foundation. Combina el Plane Tracking, Image Tracking i interacció amb contingut 3D en temps real per convertir qualsevol entorn físic pla en un camp de batalla medieval interactiu. Els jugadors ancoren un castell 3D a l'espai real i despleguen canons defensius sobre la superfície per derrotar les onades d'esquelets.

Element	Detall
Plataforma	Android / iOS (AR mòbil)
Versió	Unity 6 (6000.4.9f1)
Framework d'AR	AR Foundation 6.4.3 (ARCore / ARKit)

2. Objectiu de l'aplicació

L'objectiu principal és protegir el castell de les onades progressives d'esquelets invasors al llarg de 5 rondes de dificultat creixent, gestionant de manera tàctica el desplegament de canons defensius, limitat per recursos, sobre plans del món real.

2.1 Regles de joc i condicions de victòria

- **Defensar la fortalesa:** El castell comença amb 100 PV. Els esquelets normals són més ràpids però inflingeixen
- **Condicció de victòria:** eliminar totes les onades enemigues de les 5 rondes sense que els PV del castell arribin a 0.
- **Condicció de derrota:** si els PV del castell arriben a 0, la fortalesa cau i s'activa una pantalla de derrota amb opcions de reinici.

2.2 Cicle de recursos i economia

- **Capital inicial:** 10 monedes d'or a l'inici de la sessió.
- **Cost de desplegament dels canons:** 5 monedes per canó desplegat tocant un pla.
- **Dividend per ronda superada:** +5 monedes en sobreviure a cada ronda.

3. Funcionalitats d'AR utilitzades

3.1 AR Tracked Image Manager (seguiment d'imatges)

El joc utilitza l'AR Tracked Image Manager de Unity i una biblioteca de referència 2D

precompilada (Castle_ImageTracking.jpeg).

- **Ancoratge físic fix:** el prefab 3D del Castell s'instancia i s'ancora estrictament a la posició i orientació del món físic del marcador.
- **Origen de coordenades:** la posició del marcador és el centre espacial cap al qual es calculen tots els vectors de navegació dels enemics i els anells d'aparició radials.
- **Reinici dinàmic del cicle de vida:** ARImageReset.cs permet als jugadors eliminar les entitats rastrejades i recalibrar sense reiniciar l'aplicació.

3.2 AR Plane Manager i Raycast Manager

El joc utilitza AR Plane Manager amb seguiment de polígons horitzontals i ARRaycastManager per a la col·locació espacial.

- **Comprensió de l'entorn:** els terres i les taules del món real s'escanegen en temps real, i es projecten malles visuals dels seus límits.
- **Raigs de pantalla a món:** s'intercepten els tocs a la pantalla i es llancen raigs contra els plans físics detectats per col·locar els canons defensius.
- **Comprovacions de validació:** la col·locació es valida segons les regles d'economia (CoinManager) i l'estabilitat de la superfície abans d'instanciar.

3.3 Interfície espacial d'AR i orientació cap a la càmera (billboarding)

L'AR mòbil requereix una retroalimentació dins del món que continuï essent llegible mentre el jugador es mou per l'habitació. El sistema integra Billboard.cs i el seguiment dinàmic de la càmera a SkeletonBehavior.LateUpdate(), que alinea les barres de vida 3D flotants de TextMeshPro perquè mirin sempre cap a Camera.main.transform, independentment de l'angle de visió.

4. Aspecte innovador

Els jocs d'AR tradicionals es basen només en la detecció de plans, que pateix una deriva espacial acumulada, o només en marcadors d'imatge, que limiten la interacció a una targeta petita. AR Castle Defense unifica ambdós paradigmes: la fortalesa central utilitza el seguiment per marcador per obtenir estabilitat espacial, mentre que les defenses col·locades pel jugador utilitzen raigs lliures sobre superfícies a escala d'habitació.

5. Resum del procés de desenvolupament

El desenvolupament va seguir un enfocament iteratiu, basat en fites, amb control de versions Git i compilacions mòbils URP de Unity 6. El projecte va avançar per 5 fites principals, des d'un prototip base fins a una versió final completa.

5.1 Fites

Fita	Implementacions principals	Detalls tècnics i scripts
------	----------------------------	---------------------------

Fase 1: configuració i base	Configuració del projecte, pipeline URP, paquets XR, configuració de Git	AR Foundation 6.4.3, ARCore / ARKit, XR Interaction Toolkit, .gitignore per a LFS
Builds 1.0 i 2.0: ancoratge espacial	Transició del prototip de toc sobre plans al seguiment òptic d'imatges	ReferencImageLibrary, targeta marcador, raigs sobre plans tàctils per als canons
Build 3.0: generadors i IA	Generador radial d'enemics i IA de navegació direccional	EnemySpawner.cs (distribució radial en cercle), SkeletonBehavior.cs (vector frontal cap al castell)
Build 4.0: combat i mecàniques	Adquisició d'objectius pels canons, balística, sistemes de vida i estats de victòria i derrota	CannonBehavior.cs (rotació, comprovació d'obstacles), Bullet.cs , CastleBehavior.cs (vida i lògica de rondes)
Build 5.0: economia i polític de la IU	Gestor d'economia, límit de canons, variants d'enemics, flux complet d'IU	CoinManager.cs, CannonSpawnValidator.cs, GameManager.cs, barres de vida billboard, esquelets d'espasa

5.2 Visió general de l'arquitectura: scripts principals

Script	Responsabilitat principal	Mètodes / API clau
CastleBehavior.cs	Fa el seguiment de la salut de la fortalesa, gestiona les 5 rondes d'onades, escala la dificultat i atorga recompenses en monedes	TakeDamage(), UpdateHealth(), End(), Reset()

CannonBehavior.cs	Escaneja els enemics propers, avalua la línia de visió amb obstacles, gira la torreta amb suavitat i dispara projectils	HasObstacleInBetween(), Shoot(), OnTriggerEnter()
CoinManager.cs	Controlador d'economia central (singleton): porta el saldo de monedes, valida les col·locacions i aplica el límit de 3 canons	CanSpawnCannon(), RegisterCannonSpawned()
EnemySpawner.cs	Genera enemics de manera radial al voltant del castell; tria entre esquelets estàndard i esquelets d'espasa blindats	SpawnSkeletonAroundCastle(), ChooseEnemy()
SkeletonBehavior.cs	Es dirigeix cap al castell, causa dany en col·lisionar, gestiona els PV i el text de vida billboard orientat a la càmera	TakeDamage(), Die(), LateUpdate() (Billboard)
GameUIManager.cs	Singleton que gestiona tot el cicle de vida de la IU: menú principal, finestra d'instruccions,	UpdateWaveInfo(), ShowGameOver(), Reset()

6. Problemes trobats i solucions aplicades

Primer, les rotacions anòmales i la desalineació del canó causades per un pivot descentrat es van corregir reestructurant la jerarquia del prefab amb un pare buit centrat i restringint el gir a l'eix vertical suau ([Vector3.up](#)).

El segon problema ha sigut la inestabilitat, tremolors i duplicacions del castell provocats per pèrdues de seguiment a l'ARTrackedImageManager. Es van solucionar mitjançant un script de reinici (ARImageReset.cs) que neteja buffers, elimina instàncies obsoletes i reancora l'escena.

I el tercer problema ha sigut la il·legibilitat del text de vida dels enemics des de diferents angles. Es va resoldre aplicant un comportament de tipus billboard (Billboard.cs / LateUpdate()) que orienta contínuament els indicadors perpendicularment a la càmera del dispositiu.

7. Contribució de cada membre de l'equip

Pau Gutsens: responsable d'UI / UX, integració de sistemes

- **Àmbits de treball:** disseny i flux de la IU / HUD; scripts de joc en C#; arquitectura i comentaris de codi; localització i polit de textos; proves en dispositiu d'AR.
- **Scripts / components assignats:** GameUIManager.cs, activadors d'esdeveniments d'IU, integració de l'ensamblatge.
- **Tasques concretes realitzades:** va dissenyar i implementar des de zero el flux complet d'IU (menú principal, finestra d'instruccions, HUD en partida amb barra de PV del castell, comptadors de rondes, enemics restants i bàners de final de partida / victòria). Es va encarregar dels comentaris de codi i les traduccions, va integrar els enllaços d'IU a tots els scripts de joc i va realitzar proves en dispositius mòbils.

Martí Mira: sistemes d'economia, arquitecte del projecte

- **Àmbits de treball:** scripts de joc en C#; economia i equilibri del joc; integració de l'escena mestra; configuració del projecte; documentació tècnica; lògica de rotació de les torretes.
- **Scripts / components assignats:** CoinManager.cs, configuració de TowerDefense.unity, README.md, Project Settings.
- **Tasques concretes realitzades:** va inicialitzar la configuració del projecte Unity i els ajustos del pipeline. Va dissenyar i construir el sistema d'economia CoinManager complet (saldo de monedes, límit màxim de 3 canons, incentius de recompensa per ronda). Va treballar en els algorismes de rotació dels canons, va afinar l'escena mestra TowerDefense i va redactar la documentació del projecte.

Arnau Balasch: AR Foundation, generador i mecàniques d'enemics

- **Àmbits de treball:** seguiment i ancoratge d'imatges d'AR; IA d'enemics i aparició radial; prefabs 3D i alineació de pivots; sistema de tret dels canons; Git ignore i control de versions; variants d'enemics (esquelet d'espasa).
- **Scripts / components assignats:** EnemySpawner.cs, SkeletonBehavior.cs, CannonSpawnValidator.cs, biblioteca d'imatges de referència.
- **Tasques concretes realitzades:** va construir els prototips inicials de les builds 1.0 i 2.0 integrant el seguiment d'imatges d'AR. Va dissenyar l'EnemySpawner radial i el SkeletonBehavior amb barres de vida billboard. Va corregir els problemes d'alineació del pivot 3D del canó, va redactar la lògica de llançament de projectils i va integrar nous arquetips d'enemics (esquelets d'espasa amb escalat de vida).

Marc Blánquez: IA de combat, balística i condicions de victòria

- **Àmbits de treball:** IA de combat dels canons; raigs de línia de visió; condicions de victòria i derrota; escalat de la progressió d'onades; sistemes de dany i col·lisions; proves de joc en AR.
- **Scripts / components assignats:** CannonBehavior.cs, Bullet.cs, CastleBehavior.cs (integració de victòria / derrota).
- **Tasques concretes realitzades:** va implementar la IA d'objectius de combat dels canons i el sistema de raigs d'obstacles de línia de visió per evitar disparar a través de barreres. Va desenvolupar la lògica de rondes de CastleBehavior, els multiplicadors de dificultat progressiva i l'avaluació dels estats de victòria / derrota. Va realitzar proves de col·lisió entre els projectils i els col·lisionadors dels enemics.